

Objetivo

Resolver escenarios administrativos de Kubernetes bajo una restricción estricta de tiempo. Integrar mantenimiento de nodos, RBAC, scheduling, networking, almacenamiento y troubleshooting. Aplicar una estrategia de examen basada en lectura, priorización, ejecución y verificación. Trabajar con recursos existentes y defectuosos sin depender de instrucciones paso a paso. Validar cada solución mediante criterios técnicos objetivos antes de darla por terminada.

Requisitos previos

- Haber completado las prácticas 1 a 8 y sus prácticas complementarias.
- Tener Docker Desktop, Minikube y kubectl disponibles.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Conservar un clúster Minikube de tres nodos.
- Tener dominio operativo de kubectl, YAML, Services, RBAC, almacenamiento y troubleshooting.

Duración estimada

120 minutos

Introducción

Esta práctica es una simulación de examen inspirada en el formato práctico del Certified Kubernetes Administrator. Dispondrás de 120 minutos para resolver diez escenarios independientes con un valor total de 100 puntos. Los enunciados especifican el estado final requerido, pero no proporcionan la solución. Deberás decidir qué comandos, manifiestos y técnicas utilizar, validar cada tarea y administrar tu tiempo como si estuvieras en una evaluación práctica.

Instrucciones

Simulación tipo CKA

Información de la evaluación

Elemento	Valor
Tiempo total	120 minutos
Tareas	10
Puntuación máxima	100 puntos
Modalidad	Práctica / terminal
Clúster	Minikube de 3 nodos
Herramienta principal	kubectl
Soluciones durante el examen	No disponibles

IMPORTANTE: El examen CKA oficial es actualmente una evaluación práctica de dos horas. Esta simulación reproduce el enfoque de resolución por terminal y la presión de tiempo, pero utiliza Minikube y recursos preparados específicamente para este curso.



Reglas del modo examen

Durante los 120 minutos:

1. Lee completamente el enunciado antes de modificar recursos.
2. Verifica siempre el namespace indicado.
3. Conserva exactamente los nombres solicitados.
4. No elimines recursos que el enunciado no autorice eliminar.
5. Puedes utilizar comandos imperativos, YAML o una combinación de ambos.
6. Puedes utilizar `--dry-run=client -o yaml` para generar manifiestos.
7. Verifica cada tarea antes de avanzar.
8. Si una tarea te bloquea durante varios minutos, márcala y continúa.

9. No ejecutes el script de evaluación final hasta terminar o decidir cerrar el examen.
10. No utilices los laboratorios anteriores como soluciones literales; interpreta el escenario.



Estrategia recomendada

Una estrategia eficiente para esta simulación es:

```
0-5 min      Leer las diez tareas y marcar
              dificultad.
5-95 min     Resolver tareas priorizando puntos
              rápidos.
95-110 min   Regresar a tareas pendientes.
110-120 min  Ejecutar verificaciones y corregir
              detalles.
```

[Copiar](#)

Utiliza una hoja simple:

```
T1  [ ] 15 pts
T2  [ ] 10 pts
T3  [ ]  8 pts
T4  [ ]  7 pts
T5  [ ] 10 pts
T6  [ ] 10 pts
T7  [ ] 10 pts
T8  [ ] 10 pts
T9  [ ] 10 pts
T10 [ ] 10 pts
```

[Copiar](#)

Preparación del entorno — NO inicia el cronómetro

Preparación 1. Verificar el clúster

- **Paso 1.** Confirma que Minikube tiene tres nodos disponibles.

```
minikube status  
kubectl get nodes -o wide
```

 Copiar

El resultado debe incluir:

```
minikube  
minikube-m02  
minikube-m03
```

 Copiar

Preparación 2. Verificar Ingress y almacenamiento

- **Paso 2.** Asegura que el Ingress Controller y el provisioner de almacenamiento estén disponibles antes de iniciar.

```
minikube addons enable ingress  
minikube addons enable storage-provisioner  
minikube addons enable default-storageclass
```

 Copiar

- **Paso 3.** Espera a que el controlador Ingress quede disponible.

```
kubectl rollout status \  
  deployment/ingress-nginx-controller \  
  -n ingress-nginx \  
  --timeout=180s
```

 Copiar

Preparación 3. Crear directorio

- **Paso 4.** Crea el directorio de trabajo.

```
mkdir -p /c/LABS/kubernetes/lab9  
cd /c/LABS/kubernetes/lab9
```

 Copiar

Preparación 4. Crear alias de sesión

- **Paso 5.** Define un alias temporal para reducir escritura durante el examen.

```
alias k=kubectl
```

 Copiar

NOTA: El alias se mantiene únicamente en la terminal actual. Si abres otra terminal, deberás volver a ejecutarlo.

Preparación 5. Crear escenarios iniciales

- **Paso 6.** Crea el archivo `exam-setup.yaml`.

```
touch exam-setup.yaml
```

 Copiar

- **Paso 7.** Agrega el contenido siguiente sin modificar los errores intencionales.



```
apiVersion: v1
kind: Namespace
metadata:
  name: cka-admin
---
apiVersion: v1
kind: Namespace
metadata:
  name: cka-security
---
apiVersion: v1
kind: Namespace
metadata:
  name: cka-workloads
---
apiVersion: v1
kind: Namespace
metadata:
  name: cka-network
---
apiVersion: v1
kind: Namespace
metadata:
  name: cka-storage
---
apiVersion: v1
kind: Namespace
metadata:
  name: cka-troubleshoot
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: maintenance-app
  namespace: cka-admin
spec:
  replicas: 6
  selector:
    matchLabels:
      app: maintenance-app
  template:
    metadata:
      labels:
        app: maintenance-app
```

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: broken-backend
  namespace: cka-network
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: broken-backend-svc
  namespace: cka-network
spec:
  type: ClusterIP
  selector:
    app: backend-error
  ports:
    - port: 80
      targetPort: 8080
---
apiVersion: v1
kind: Pod
metadata:
  name: image-failure
  namespace: cka-troubleshoot
spec:
```

```
    containers:
      - name: app
        image: nginx:tag-does-not-exist-cka
    ---
apiVersion: v1
kind: Pod
metadata:
  name: crash-failure
  namespace: cka-troubleshoot
spec:
  containers:
    - name: app
      image: nginx:1.27-alpine
      command: ["/bin/sh", "-c", "exit 17"]
    ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dns-backend
  namespace: cka-troubleshoot
spec:
  replicas: 2
  selector:
    matchLabels:
      app: dns-backend
  template:
    metadata:
      labels:
        app: dns-backend
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
    ---
apiVersion: v1
kind: Service
metadata:
  name: dns-backend-svc
  namespace: cka-troubleshoot
spec:
  selector:
    app: dns-wrong
  ports:
```

```
- port: 80
  targetPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: dns-client
  namespace: cka-troubleshoot
spec:
  dnsPolicy: None
  dnsConfig:
    nameservers:
      - 1.2.3.4
  containers:
    - name: tools
      image: busybox:1.36
      command: ["sh", "-c", "sleep 7200"]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: production-api
  namespace: cka-troubleshoot
spec:
  replicas: 3
  selector:
    matchLabels:
      app: production-api
  template:
    metadata:
      labels:
        app: production-api
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /ready-does-not-exist
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 3
      ---
```

```
apiVersion: v1
kind: Service
metadata:
  name: production-api-svc
  namespace: cka-troubleshoot
spec:
  selector:
    app: production-api
  ports:
    - port: 80
      targetPort: 80
```

- **Paso 8.** Aplica la configuración inicial.

```
kubectl apply -f exam-setup.yaml
```

 Copiar

- **Paso 9.** Espera 20 segundos y observa únicamente el estado general, sin resolver todavía los fallos.

```
sleep 20
```

 Copiar

```
kubectl get pods -A | grep -E 'cka-|NAMESPACE'
```



INICIA EL CRONÓMETRO — 120 MINUTOS

Tarea 1 — Mantenimiento de nodo

Dominio: Cluster Architecture, Installation & Configuration

Valor: 15 puntos

Tiempo objetivo: 15 minutos

Escenario

El nodo `minikube-m02` debe entrar en mantenimiento sin dejar nuevos workloads programados sobre él.

El Deployment `maintenance-app`, ubicado en `cka-admin`, debe permanecer disponible durante la operación.

Estado final requerido

Realiza las acciones necesarias para que:

- `minikube-m02` quede marcado temporalmente como no programable.
- los workloads administrados que puedan desalojarse sean retirados del nodo;
- los DaemonSets no bloqueen la operación;
- los Pods desalojados sean reprogramados en nodos disponibles;
- al finalizar, `minikube-m02` vuelva a aceptar scheduling;
- `maintenance-app` termine con **6/6 réplicas disponibles**.

Restricciones

- No elimines el nodo.
- No elimines el Deployment.
- No reduzcas permanentemente el número de réplicas.
- No reinicies Minikube.

Criterios de aceptación

```
[ ] minikube-m02 termina en estado  
Ready,SchedulingEnabled.  
[ ] maintenance-app termina con 6 réplicas  
disponibles.  
[ ] Ningún Pod de maintenance-app queda Pending.
```

[Copiar](#)

Verificación permitida

```
kubectl get nodes  
kubectl get pods -n cka-admin -o wide  
kubectl get deployment maintenance-app -n cka-admin
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Mantenimiento de nodo completado correctamente, conservando disponibilidad de workloads y restaurando el scheduling al

finalizar.

Tarea 2 — RBAC con mínimo privilegio

Dominio: Cluster Architecture, Installation & Configuration

Valor: 10 puntos

Tiempo objetivo: 10 minutos

Escenario

Una aplicación necesita consultar información operativa en el namespace `cka-security`.

Crea los recursos necesarios para proporcionar acceso de solo lectura.

Estado final requerido

En `cka-security` deben existir:

- ServiceAccount: `auditor`
- Role: `workload-reader`
- RoleBinding: `auditor-reader`

La identidad `auditor` debe poder:

```
get, list, watch Pods  
get, list ConfigMaps
```

 Copiar

No debe poder:

```
create Pods  
delete Pods  
get Secrets
```

 Copiar

Restricciones

- No utilices `ClusterRole`.
- No utilices wildcards `*`.
- No concedas acceso a Secrets.

- El permiso debe quedar limitado a `cka-security`.

Criterios de aceptación

```
[ ] ServiceAccount auditor existe.  
[ ] Role workload-reader contiene solo los permisos  
requeridos.  
[ ] RoleBinding auditor-reader vincula auditor con  
workload-reader.  
[ ] kubectl auth can-i get pods devuelve yes.  
[ ] kubectl auth can-i get secrets devuelve no.
```

[Copiar](#)

Comando de identidad para tus pruebas

```
system:serviceaccount:cka-security:auditor
```

[Copiar](#)

Tarea finalizada

Resultado esperado: ServiceAccount, Role y RoleBinding configurados con mínimo privilegio, validando permisos permitidos y denegados mediante RBAC.

Tarea 3 — Rolling update y rollback

Dominio: Workloads & Scheduling

Valor: 8 puntos

Tiempo objetivo: 9 minutos

Escenario

Debes crear una aplicación web administrada mediante Deployment.

Estado final requerido

En `cka-workloads` crea:

```
Deployment: exam-web  
Réplicas: 4  
Imagen inicial: nginx:1.27-alpine
```

 Copiar

Después:

1. actualiza la imagen a `nginx:1.28-alpine` ;
2. verifica el rollout;
3. revierte a la revisión anterior.

El estado final debe ser:

```
4/4 réplicas Ready  
imagen nginx:1.27-alpine  
historial con al menos dos revisiones
```

 Copiar

Restricciones

- Debes utilizar el mecanismo de rollout del Deployment.
- No resuelvas el rollback eliminando y recreando el Deployment.

Criterios de aceptación

```
[ ] exam-web existe con 4 réplicas.  
[ ] El historial contiene más de una revisión.  
[ ] La imagen final es nginx:1.27-alpine.  
[ ] El Deployment finaliza 4/4 Ready.
```

 Copiar

Tarea finalizada

Resultado esperado: Deployment actualizado y revertido mediante rollout, conservando cuatro réplicas disponibles y la imagen final solicitada.

Tarea 4 — Scheduling con label, taint y toleration

Dominio: Workloads & Scheduling

Valor: 7 puntos

Tiempo objetivo: 9 minutos

Escenario

El nodo `minikube-m03` será reservado para una carga analítica.

Estado final requerido

Configura el nodo con:

```
label:  
workload=analytics  
  
taint:  
dedicated=analytics:NoSchedule
```

 Copiar

Después crea en `cka-workloads`:

```
Deployment: analytics-app  
Réplicas: 2  
Imagen: nginx:1.27-alpine
```

 Copiar

Los Pods de `analytics-app` deben:

- ejecutarse exclusivamente en `minikube-m03`;
- tolerar el taint configurado;
- terminar `Running` y `Ready`.

Restricciones

- No elimines el taint para hacer funcionar la aplicación.
- No utilices `nodeName`.
- Debes utilizar scheduling declarativo.

Criterios de aceptación

```
[ ] minikube-m03 tiene el label workload=analytics.  
[ ] minikube-m03 tiene el taint  
dedicated=analytics:NoSchedule.  
[ ] analytics-app tiene 2/2 réplicas Ready.  
[ ] Ambos Pods ejecutan en minikube-m03.
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Workload analítico programado exclusivamente en el nodo designado mediante labels, taints, tolerations y scheduling declarativo.

Tarea 5 — Reparar Service y exponer mediante NodePort

Dominio: Services & Networking

Valor: 10 puntos

Tiempo objetivo: 11 minutos

Escenario

El Deployment `broken-backend` está operativo, pero el Service `broken-backend-svc` no permite acceder a sus Pods.

Estado final requerido

Diagnostica y corrige el Service existente.

Después conviértelo o configúralo para cumplir:

```
Nombre: broken-backend-svc  
Namespace: cka-network  
Tipo: NodePort  
Service port: 80  
Target port: 80  
NodePort: 30081
```

[Copiar](#)

Restricciones

- No modifiques los labels del Deployment.
- No elimines `broken-backend`.
- Conserva el nombre del Service.

Criterios de aceptación

```
[ ] El selector del Service coincide con los Pods.  
[ ] El targetPort es 80.  
[ ] El tipo del Service es NodePort.  
[ ] nodePort es exactamente 30081.  
[ ] El Service tiene 2 endpoints disponibles.
```

[Copiar](#)

Validación funcional

En Windows/macOS con driver Docker puedes utilizar:

```
minikube service broken-backend-svc \  
-n cka-network \  
--url
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Service defectuoso reparado y expuesto mediante NodePort, recuperando endpoints y conectividad hacia los Pods backend.

Tarea 6 — Ingress con enrutamiento por path

Dominio: Services & Networking

Valor: 10 puntos

Tiempo objetivo: 12 minutos

Escenario

Debes publicar dos aplicaciones detrás de un único Ingress.

Estado final requerido

En `cka-network` crea:

Aplicación frontend

```
Deployment: portal
Réplicas: 2
Imagen: nginx:1.27-alpine
Service: portal-svc
Service port: 80
```

[Copiar](#)

Aplicación API

```
Deployment: api
Réplicas: 2
Imagen: nginx:1.27-alpine
Service: api-svc
Service port: 80
```

[Copiar](#)

Ingress

```
Nombre: exam-ingress
IngressClass: nginx

/      → portal-svc:80
/api   → api-svc:80
```

[Copiar](#)

La ruta `/api` debe llegar correctamente al backend aunque NGINX solo disponga de `/` como contenido inicial.

Restricciones

- No expongas los Deployments mediante NodePort.
- Utiliza Services ClusterIP.
- Conserva exactamente los nombres solicitados.

Criterios de aceptación

```
[ ] portal tiene 2/2 réplicas Ready.  
[ ] api tiene 2/2 réplicas Ready.  
[ ] portal-svc y api-svc tienen endpoints.  
[ ] exam-ingress utiliza la clase nginx.  
[ ] / apunta a portal-svc.  
[ ] /api apunta a api-svc.
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Ingress configurado correctamente para enrutar `/` y `/api` hacia Services independientes con backends disponibles.

Tarea 7 — PersistentVolume y PersistentVolumeClaim

Dominio: Storage

Valor: 10 puntos

Tiempo objetivo: 12 minutos

Escenario

Una aplicación necesita almacenamiento persistente estático.

Estado final requerido

Crea:

PersistentVolume

```
Nombre: exam-pv
Capacidad: 1Gi
AccessMode: ReadWriteOnce
ReclaimPolicy: Retain
StorageClassName: ""
Backend: hostPath
Ruta: /mnt/data/cka-exam
Nodo permitido: minikube
```

 Copiar

PersistentVolumeClaim

En `cka-storage`:

```
Nombre: exam-pvc
Solicitud: 500Mi
AccessMode: ReadWriteOnce
StorageClassName: ""
Debe enlazarse con exam-pv
```

 Copiar

Pod consumidor

```
Nombre: storage-check
Namespace: cka-storage
Imagen: busybox:1.36
MountPath: /data
```

 Copiar

El Pod debe permanecer en ejecución y debe existir:

```
/data/exam.txt
```

 Copiar

con el contenido exacto:

```
CKA-STORAGE-OK
```

 Copiar

Criterios de aceptación

```
[ ] exam-pv está Bound.  
[ ] exam-pvc está Bound con exam-pv.  
[ ] storage-check está Running.  
[ ] storage-check ejecuta en minikube.  
[ ] /data/exam.txt contiene CKA-STORAGE-OK.
```

[Copiar](#)

Tarea finalizada

Resultado esperado: PersistentVolume, PersistentVolumeClaim y Pod consumidor configurados correctamente, validando binding y persistencia del dato solicitado.

Tarea 8 — Troubleshooting de Pods

Dominio: Troubleshooting

Valor: 10 puntos

Tiempo objetivo: 10 minutos

Escenario

En `cka-troubleshoot` existen dos Pods defectuosos:

```
image-failure  
crash-failure
```

[Copiar](#)

Estado final requerido

Diagnostica ambos recursos y corrige la causa raíz.

Al finalizar:

```
image-failure → Running / Ready  
crash-failure → Running / Ready
```

[Copiar](#)

Ambos deben utilizar:

`nginx:1.27-alpine` Copiar

Restricciones

- Antes de corregir cada Pod debes revisar la evidencia disponible mediante `describe`, `logs` o Events.
- Conserva los nombres originales.
- No reemplaces los Pods por Deployments.

Criterios de aceptación

```
[ ] image-failure utiliza nginx:1.27-alpine.  
[ ] image-failure está Running.  
[ ] crash-failure utiliza nginx:1.27-alpine.  
[ ] crash-failure permanece Running sin reinicios  
continuos.
```

 Copiar

Tarea finalizada

Resultado esperado: Pods con fallos de imagen y ejecución diagnosticados y corregidos hasta alcanzar estado `Running` y `Ready`.

Tarea 9 — Service y DNS

Dominio: Troubleshooting

Valor: 10 puntos

Tiempo objetivo: 10 minutos

Escenario

En `cka-troubleshoot`:

- `dns-backend` tiene dos Pods saludables;
- `dns-backend-svc` no encuentra correctamente sus backends;
- `dns-client` no utiliza la política DNS normal del clúster.

Estado final requerido

Corrige ambos problemas.

Desde `dns-client` deben funcionar:

```
nslookup dns-backend-svc  
  
nslookup dns-backend-svc.cka-  
troubleshoot.svc.cluster.local
```

[Copiar](#)

y la petición:

```
http://dns-backend-svc
```

[Copiar](#)

debe devolver contenido NGINX.

Restricciones

- No modifiques CoreDNS.
- No reinicies el clúster.
- No cambies los labels del Deployment `dns-backend`.
- Conserva los nombres de los recursos.

Criterios de aceptación

```
[ ] dns-backend-svc tiene 2 endpoints.  
[ ] dns-client utiliza la política DNS normal del  
clúster.  
[ ] El nombre corto resuelve.  
[ ] El FQDN resuelve.  
[ ] La petición HTTP al Service funciona.
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Service y resolución DNS restaurados correctamente, validando endpoints, nombre corto, FQDN y acceso HTTP interno.

Tarea 10 — Incidente integral de disponibilidad

Dominio: Troubleshooting

Valor: 10 puntos

Tiempo objetivo: 12 minutos

Escenario

El Service `production-api-svc` existe y el Deployment `production-api` mantiene tres Pods en ejecución, pero la aplicación no está disponible a través del Service.

Tu responsabilidad es encontrar la causa raíz y restaurar el servicio.

Estado final requerido

Al finalizar:

```
Deployment production-api:  
3/3 réplicas Ready
```

[Copiar](#)

```
Service production-api-svc:  
3 endpoints disponibles
```

Desde un Pod temporal dentro de `cka-troubleshoot`:

```
wget http://production-api-svc
```

[Copiar](#)

debe devolver la página de NGINX.

Restricciones

- No elimines el Deployment.
- No elimines el Service.
- No reduzcas las réplicas.
- No cambies la imagen.
- Corrige únicamente la configuración responsable del incidente.

Criterios de aceptación

```
[ ] production-api tiene 3/3 réplicas Ready.  
[ ] Los Pods no presentan reinicios continuos.  
[ ] production-api-svc tiene 3 endpoints.  
[ ] La prueba HTTP devuelve contenido correctamente.
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Incidente integral de disponibilidad resuelto, recuperando tres réplicas Ready, endpoints activos y conectividad funcional al Service.



FIN DEL EXAMEN

Cuando el cronómetro llegue a cero:

1. Detén cualquier modificación.
2. No borres recursos fallidos.
3. Ejecuta únicamente las verificaciones siguientes.
4. Registra tu puntuación.



Evaluación automática

Crear el evaluador

- **Paso 10.** Crea `validate-exam.sh` con el contenido siguiente.

```
cat > validate-exam.sh <<'EOF'
#!/usr/bin/env bash

PASS=0
TOTAL=100

pass() {
    echo "✅ $1 (+$2)"
    PASS=$((PASS + $2))
}

fail() {
    echo "❌ $1 (+0/$2)"
}

echo
"=====
="
echo "          EVALUACION SIMULACION TIPO CKA"
echo
"=====
="

echo ""
echo "--- T1 Mantenimiento de nodo: 15 puntos ---"

UNCHED=$(kubectl get node minikube-m02 \
    -o jsonpath='{.spec.unschedulable}'
2>/dev/null)

READY=$(kubectl get deployment maintenance-app \
    -n cka-admin \
    -o jsonpath='{.status.readyReplicas}'
2>/dev/null)

if [ "$UNCHED" != "true" ] && [ "$READY" = "6" ]
]; then
    pass "T1 mantenimiento completado" 15
else
    fail "T1 mantenimiento incompleto" 15
fi

echo ""
```

 Copiar

```
echo "--- T2 RBAC: 10 puntos ---"

CAN_GET=$(kubectl auth can-i get pods \
  --as=system:serviceaccount:cka-
security:auditor \
  -n cka-security 2>/dev/null)

CAN_SECRET=$(kubectl auth can-i get secrets \
  --as=system:serviceaccount:cka-
security:auditor \
  -n cka-security 2>/dev/null)

if [ "$CAN_GET" = "yes" ] && [ "$CAN_SECRET" =
"no" ]; then
  pass "T2 RBAC correcto" 10
else
  fail "T2 RBAC incorrecto" 10
fi

echo ""
echo "--- T3 Rollout: 8 puntos ---"

T3_READY=$(kubectl get deployment exam-web \
  -n cka-workloads \
  -o jsonpath='{.status.readyReplicas}'
2>/dev/null)

T3_IMAGE=$(kubectl get deployment exam-web \
  -n cka-workloads \
  -o
jsonpath='{.spec.template.spec.containers[0].ima
ge}' 2>/dev/null)

if [ "$T3_READY" = "4" ] && [ "$T3_IMAGE" =
"nginx:1.27-alpine" ]; then
  pass "T3 rollout y rollback correctos" 8
else
  fail "T3 rollout incompleto" 8
fi

echo ""
echo "--- T4 Scheduling: 7 puntos ---"

T4_READY=$(kubectl get deployment analytics-app
\
```

```
-n cka-workloads \
-o jsonpath='{.status.readyReplicas}'
2>/dev/null)

T4_NODES=$(kubectl get pod \
-n cka-workloads \
-l app=analytics-app \
-o jsonpath='{.items[*].spec.nodeName}'
2>/dev/null)

if [ "$T4_READY" = "2" ] && \
    echo "$T4_NODES" | grep -q "minikube-m03" && \
    ! echo "$T4_NODES" | grep -q "minikube-m02";
then
    pass "T4 scheduling correcto" 7
else
    fail "T4 scheduling incorrecto" 7
fi

echo ""
echo "--- T5 Service NodePort: 10 puntos ---"

T5_TYPE=$(kubectl get service broken-backend-svc \
-n cka-network \
-o jsonpath='{.spec.type}' 2>/dev/null)

T5_NODEPORT=$(kubectl get service broken-backend-svc \
-n cka-network \
-o jsonpath='{.spec.ports[0].nodePort}'
2>/dev/null)

T5_EP=$(kubectl get endpoints broken-backend-svc \
-n cka-network \
-o jsonpath='{.subsets[0].addresses[*].ip}'
2>/dev/null | wc -w)

if [ "$T5_TYPE" = "NodePort" ] && \
    [ "$T5_NODEPORT" = "30081" ] && \
    [ "$T5_EP" -eq 2 ]; then
    pass "T5 Service reparado" 10
else
```

```
fail "T5 Service incorrecto" 10
fi

echo ""
echo "--- T6 Ingress: 10 puntos ---"

PORTAL_READY=$(kubectl get deployment portal \
  -n cka-network \
  -o jsonpath='{.status.readyReplicas}'
2>/dev/null)

API_READY=$(kubectl get deployment api \
  -n cka-network \
  -o jsonpath='{.status.readyReplicas}'
2>/dev/null)

INGRESS_CLASS=$(kubectl get ingress exam-ingress \
  -n cka-network \
  -o jsonpath='{.spec.ingressClassName}'
2>/dev/null)

if [ "$PORTAL_READY" = "2" ] && \
  [ "$API_READY" = "2" ] && \
  [ "$INGRESS_CLASS" = "nginx" ]; then
  pass "T6 Ingress creado" 10
else
  fail "T6 Ingress incompleto" 10
fi

echo ""
echo "--- T7 Storage: 10 puntos ---"

PVC_PHASE=$(kubectl get pvc exam-pvc \
  -n cka-storage \
  -o jsonpath='{.status.phase}' 2>/dev/null)

STORAGE_DATA=$(kubectl exec storage-check \
  -n cka-storage -- \
  cat /data/exam.txt 2>/dev/null)

if [ "$PVC_PHASE" = "Bound" ] && \
  [ "$STORAGE_DATA" = "CKA-STORAGE-OK" ]; then
  pass "T7 almacenamiento correcto" 10
else
```

```
fail "T7 almacenamiento incorrecto" 10
fi

echo ""
echo "--- T8 Troubleshooting Pods: 10 puntos ---"

P1=$(kubectl get pod image-failure \
  -n cka-troubleshoot \
  -o jsonpath='{.status.phase}' 2>/dev/null)

P2=$(kubectl get pod crash-failure \
  -n cka-troubleshoot \
  -o jsonpath='{.status.phase}' 2>/dev/null)

if [ "$P1" = "Running" ] && [ "$P2" = "Running" ]; then
  pass "T8 Pods reparados" 10
else
  fail "T8 Pods continúan fallando" 10
fi

echo ""
echo "--- T9 DNS y Service: 10 puntos ---"

DNS_EP=$(kubectl get endpoints dns-backend-svc \
  -n cka-troubleshoot \
  -o jsonpath='{.subsets[0].addresses[*].ip}'
2>/dev/null | wc -w)

kubectl exec dns-client \
  -n cka-troubleshoot -- \
  nslookup dns-backend-svc >/dev/null 2>&1
DNS_OK=$?

if [ "$DNS_EP" -eq 2 ] && [ "$DNS_OK" -eq 0 ];
then
  pass "T9 DNS y Service correctos" 10
else
  fail "T9 DNS o Service incorrectos" 10
fi

echo ""
echo "--- T10 Incidente integral: 10 puntos ---"
```

```

PROD_READY=$(kubectl get deployment production-
api \
  -n cka-troubleshoot \
  -o jsonpath='{.status.readyReplicas}'
2>/dev/null)

PROD_EP=$(kubectl get endpoints production-api-
svc \
  -n cka-troubleshoot \
  -o jsonpath='{.subsets[0].addresses[*].ip}'
2>/dev/null | wc -w)

if [ "$PROD_READY" = "3" ] && [ "$PROD_EP" -eq 3
]; then
  pass "T10 disponibilidad restaurada" 10
else
  fail "T10 incidente no resuelto" 10
fi

echo ""
echo
"=====
="
echo "PUNTUACION FINAL: $PASS / $TOTAL"
echo
"=====
="

if [ "$PASS" -ge 75 ]; then
  echo " 🎓 RESULTADO PEDAGOGICO: APROBADO"
else
  echo " 📚 RESULTADO PEDAGOGICO: REQUIERE
REPASO"
fi
EOF

```

- **Paso 11.** Ejecuta el evaluador.

```

chmod +x validate-exam.sh
./validate-exam.sh

```

 Copiar

IMPORTANTE: El umbral de 75 puntos utilizado aquí es una regla pedagógica de esta práctica. No debe interpretarse como el puntaje oficial vigente del examen CKA.



Interpretación pedagógica del resultado

Puntuación	Interpretación
90–100	Excelente dominio operativo y buena administración del tiempo
80–89	Preparación sólida; revisar errores puntuales
75–79	Aprobación pedagógica mínima; todavía existen áreas de riesgo
60–74	Conocimiento funcional, pero insuficiente bajo presión de tiempo
<60	Requiere repasar fundamentos antes de realizar otra simulación



Hoja de revisión posterior

Después de finalizar el cronómetro y registrar la puntuación, documenta:

Tareas completadas sin ayuda:
Tareas completadas después de regresar:
Tareas no completadas:
Tarea que consumió más tiempo:
Comandos que necesitaste consultar:
Errores por namespace:
Errores de sintaxis YAML:
Errores de interpretación:
Puntuación final:
Tiempo utilizado:

 Copiar

La finalidad de esta hoja es detectar si el problema fue de conocimiento, sintaxis, diagnóstico o administración del tiempo.

Troubleshooting de la simulación

Problema 1. El entorno inicial no coincide con los escenarios

Si un recurso de preparación fue modificado accidentalmente antes de iniciar, restablece únicamente el escenario correspondiente desde `exam-setup.yaml`.

```
kubectl apply -f exam-setup.yaml
```

 Copiar

No ejecutes esta acción durante la evaluación si ya comenzaste a resolver tareas, porque podría sobrescribir cambios válidos.

Problema 2. El Ingress Controller no está disponible

Comprueba su estado:

```
kubectl get pods -n ingress-nginx  
kubectl get ingressclass
```

 Copiar

La preparación del entorno debe completarse antes de iniciar el cronómetro.

Problema 3. Un NodePort no es accesible directamente desde Windows o macOS

Esto no implica necesariamente un error del Service con el driver Docker.

Utiliza:

```
minikube service NOMBRE \  
-n NAMESPACE \  
--url
```

[Copiar](#)

y conserva abierta la terminal si Minikube necesita mantener un túnel local.

Problema 4. T4 deja cargas ajenas sin programar

El taint `dedicated=analytics:NoSchedule` afecta a nuevos Pods que no lo toleren.

Esto forma parte del escenario. No elimines el taint durante el examen porque es un criterio explícito de aceptación.

Problema 5. El PV de T7 queda Available y el PVC Pending

Revisa compatibilidad entre:

```
storageClassName  
accessModes  
capacity  
volumeName  
nodeAffinity
```

[Copiar](#)

No cambies aleatoriamente varios parámetros al mismo tiempo; identifica primero cuál impide el binding.



Limpieza posterior

Ejecuta la limpieza solo después de registrar el resultado.

```
kubectl delete namespace \
  cka-admin \
  cka-security \
  cka-workloads \
  cka-network \
  cka-storage \
  cka-troubleshoot \
  --ignore-not-found=true

kubectl delete pv exam-pv \
  --ignore-not-found=true

kubectl uncordon minikube-m02 \
  2>/dev/null || true

kubectl taint node minikube-m03 \
  dedicated=analytics:NoSchedule- \
  2>/dev/null || true

kubectl label node minikube-m03 \
  workload- \
  2>/dev/null || true

kubectl config set-context \
  --current \
  --namespace=default
```

 Copiar

No elimines el clúster Minikube si deseas repetir la simulación.



Qué evalúa esta simulación

La distribución se diseñó para aproximarse a los dominios actuales del CKA dentro de lo que puede reproducirse razonablemente en un clúster Minikube local:

Dominio	Tareas	Puntos
Cluster Architecture, Installation & Configuration	T1, T2	25
Workloads & Scheduling	T3, T4	15
Services & Networking	T5, T6	20
Storage	T7	10
Troubleshooting	T8, T9, T10	30
Total	10 tareas	100

La simulación prioriza trabajo real en terminal, validación de estado y diagnóstico bajo presión de tiempo. No intenta copiar preguntas del examen oficial ni sustituye el simulador oficial.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la simulación habrás resuelto diez escenarios integrales de administración de Kubernetes bajo presión de tiempo, aplicando técnicas de mantenimiento, configuración, networking, almacenamiento, scheduling, RBAC y troubleshooting.

Notas y consideraciones

- Esta simulación utiliza Minikube y no reproduce la plataforma oficial de entrega del examen CKA.
- La preparación del entorno se realiza antes de iniciar el cronómetro.
- Durante el modo examen no se incluyen prompts de apoyo con IA ni soluciones paso a paso.
- Cada tarea es independiente; si una tarea consume demasiado tiempo, continúa con la siguiente y regresa al final.
- La puntuación de esta práctica es pedagógica y no representa el sistema oficial de calificación del CKA.

URL de referencia

- [Certified Kubernetes Administrator \(CKA\)](#)
- [kubect! Cheat Sheet](#)
- [Debug Running Pods](#)

- [Debug Services](#)